[Trung Nguyen] Session 9: Linux User-Kernel Communication

1. Overview of user-kernel communication interfaces

User programs run in **user space** with no direct access to hardware or kernel memory. Anything that needs the kernel (files, devices, network, process control) must cross the user/kernel boundary through an interface the kernel provides.

There is only one real gate: the **system call**. The other interfaces are different ways of *using* system calls:

overview

- **System call**: a fixed, typed function for a general OS service.
- **ioctl**: one generic syscall that forwards driver-specific commands on an fd.
- **/proc**: kernel data exposed as virtual text files, read/written with normal file I/O.
- **Netlink**: a socket carrying structured messages in both directions, including kernel-initiated events.

1.1 System Call, IOCTL, proc file system, Netlink Socket - what each is for and when to choose it

	System call	ioctl	/proc (and /sys)	Netlink
Purpose	Core OS services	Device/driver control	Inspect status, tune parameters	Configure subsystems, receive events
Data format	Typed C arguments	Binary struct, driver-defined	Text	Structured binary messages
Direction	User → kernel (request/reply)	User → kernel (request/reply)	User reads/writes on demand	Both ways, kernel can push
Needs	Nothing	An fd (often a device file)	A file path	A netlink socket
Typical users	Every program	`tcsetattr`, V4L2, GPIO, KVM	`ps`, `top`, `free`, `sysctl`	`ip` (iproute2), udev

When to choose:

- **System call**: a general service every program needs. You never add one for a single driver; new syscalls are rare and reviewed heavily.
- **ioctl**: you control a specific device through its fd and need fast, structured commands (set mode, get config, start DMA).
- **/proc or /sys**: simple values that humans and shell scripts should read or change with `cat`/`echo`. Use `/sys` for device/driver attributes, `/proc` for processes and kernel-wide info.
- **Netlink**: the kernel must *notify* user space of events, or you exchange complex, extensible messages (networking config, hotplug).

Rule of thumb: *one value, human-readable* → sysfs/procfs; *command to a device* → ioctl; *events or rich two-way messages* → netlink.

2. System call

A **system call** is how a user-mode process asks the kernel to do something on its behalf: open a file, read/write, create a process, get more memory. It is the only controlled entry point into the kernel.

overview system call

The **system call interface** is the set of all system calls the kernel provides, plus the convention for calling them (syscall number, how arguments are passed, how results and errors are returned). It is the boundary between user space and kernel space.

Core ideas

1. **Protection, enforced by hardware.** The CPU has two privilege levels: user mode and kernel mode. In user mode, privileged instructions and kernel memory are blocked. So only the kernel can touch hardware or sensitive data.
2. **Controlled mode switch.** A process cannot jump into the kernel freely. It executes a special trap instruction (`syscall` on x86-64, `svc` on ARM), which switches the CPU to kernel mode and jumps to a fixed entry point chosen by the kernel.
3. **The kernel validates first.** It checks the syscall number, the arguments, and the process's permissions. Data behind user pointers is copied safely (`copy_from_user` / `copy_to_user`), never trusted directly.
4. **Runs in process context.** The same process (same PID) runs kernel code on its own kernel stack. A syscall is a *mode switch*, not a *context switch*.
5. **Can block.** A syscall like `read()` on an empty pipe puts the process to sleep until data arrives; the scheduler runs other processes meanwhile. A signal may interrupt it (`EINTR`).
6. **Library function ≠ system call.** `printf()` and `malloc()` are library functions; they may make zero, one, or many syscalls. (`man 2` = syscalls, `man 3` = library functions.)

Flow

alt text

1. The program calls a C library wrapper, e.g. `read()`.
2. The wrapper puts the syscall number and arguments into registers.
3. It executes the trap instruction; the CPU enters kernel mode.
4. The kernel saves registers, looks up the handler in the **system call table**, and runs it.
5. The kernel puts the result in a register and returns to user mode. On error it returns a negative error code.
6. The wrapper turns an error into `errno` and returns `-1`; otherwise it returns the result.

Key takeaways

- Syscalls are much slower than normal function calls (mode switch, saving state, checks), so batch work into fewer calls (that is why stdio buffers).
- Linux keeps the syscall ABI stable, so old binaries keep working on new kernels. Syscall numbers differ between CPU architectures.
- `strace ./prog` shows every syscall a program makes.

3. ioctl interface

ioctl (input/output control) is a general-purpose system call for sending device-specific control commands through a file descriptor: `ioctl(fd, request, arg)`.

alt text ***Core ideas***

1. **Problem.** In UNIX "everything is a file", but `read()`/`write()` only move bytes. Devices also need operations like setting a serial port's baud rate or reading the terminal size. Adding one syscall per such operation would bloat the kernel.
2. **Solution.** One syscall acts as a common gate. The meaning of `request` and `arg` is defined by each driver; the kernel just forwards them.
3. **Mechanism.** The kernel uses the fd to find the driver and calls the driver's ioctl handler (registered in `file_operations`), which is usually a `switch (request)`. Pointer data is copied with `copy_from_user` / `copy_to_user`.
4. **Request codes** are built with the `_IO`, `_IOR`, `_IOW`, `_IOWR` macros, encoding a driver ID, command number, data direction, and data size, to avoid collisions.

Key takeaways

- There is no common standard for commands; read the driver's header or docs.
- It is hard to type-check and audit, so newer kernel interfaces often prefer sysfs or netlink, though ioctl is still widely used.

4. proc file system interface

/proc is a virtual (pseudo) file system mounted at `/proc`. It exposes kernel and process information as files you can read (and sometimes write) with normal file operations or tools like `cat` and `echo`. alt text ***Core ideas***

1. **Problem.** Programs need kernel information (memory usage, open files of a process) and sometimes need to tune kernel parameters. One syscall per item would not scale, so the kernel exposes them as files.
2. **Mechanism.** Nothing is stored on disk. When you `read()` a /proc file, the kernel calls a handler that formats the current data as text. Writing to some files calls a handler that changes a kernel parameter.
3. **Main layout:**

Path	Content
`/proc/<PID>/`	Info about one process: `status`, `cmdline`, `maps`, `fd/`, `exe`, `cwd`
`/proc/self`	Link to the current process's own directory
`/proc/cpuinfo`, `/proc/meminfo`, ...	System-wide info
`/proc/sys/`	Tunable kernel parameters (read/write)

4. **Kernel side.** A module can create its own /proc entry with `proc_create()`.

Key takeaways

- Data is text: easy for humans and shell tools, but programs must parse it, and the format may change.
- Content is a snapshot at read time.
- /proc became cluttered over time, so **sysfs** (`/sys`) was added to represent devices and drivers in a more organized way.

5. Netlink socket interface

Netlink is a two-way message channel between user space and the kernel, using the socket API with the `AF_NETLINK` address family. Data is exchanged as structured binary messages.

Core ideas

1. **Problem.** With ioctl and /proc, only user space can ask; the kernel cannot push notifications. Programs would have to poll to detect events like a cable plugged in or an IP address change.
2. **Solution.** Use sockets. This gives asynchronous send/receive, queuing, `poll()`/`select()`/`epoll()` support, and multicast groups so the kernel can broadcast events.
3. **Mechanism.**
   - Create a socket: `socket(AF_NETLINK, SOCK_RAW, NETLINK_ROUTE)`. The protocol selects the kernel subsystem (e.g. `NETLINK_ROUTE` for networking).
   - `bind()` with a `struct sockaddr_nl` to set your ID and the multicast groups to join.
   - Each message starts with a `struct nlmsghdr` header (length, type, flags, sequence number, sender ID), followed by the payload.
4. **Two usage patterns:** request/response (e.g. add an IP address) and event notifications (subscribe to a group, and the kernel sends messages when something happens).

Key takeaways

- Modern network tools like `ip` (iproute2) use netlink, replacing the old ioctl-based `ifconfig`/`route`. udev also uses it for device hotplug events.
- Structured binary data: more precise than /proc text, but must be parsed by format.
- No device file needed, and communication works in both directions.
- Messages can be dropped if the receive queue overflows, so programs must handle that.

6. Lab

[Link Git] https://github.com/trungnm2111/Doc_Session9

6.1 User-space app that controls an existing module through system calls

Goal

Write a user-space program that controls an LED. The LED is managed by a driver in kernel space. User code cannot call driver functions directly, so the program must go through the kernel using **system calls**.

In this lab the system calls are made directly with `syscall(2)` instead of the usual libc wrappers such as `open()` and `write()`. This makes every step between the app and the driver visible.

How the two sides connect

- **Driver side (provided).** `misc_register()` creates the device node `/dev/myled`. The driver registers `myled_fops`, a `struct file_operations` (a table of callback function pointers). It tells the kernel: on write, call `myled_write`; on read, call `myled_read`.
- **App side (to be written).** The app treats `/dev/myled` as an ordinary file and uses the standard sequence: open, write or read, close.
- **Bridge.** The **VFS** (Virtual File System) is the kernel layer that receives every file-related syscall and routes it to the driver that owns the file.

Flow of a command

Example: the app turns the LED on.

```
App: syscall(SYS_write, fd, "1", 1)
        ↓  (user space → kernel space)
VFS: looks up the file for fd → /dev/myled
        ↓
Kernel calls myled_write() in the driver
        ↓
Driver: copy_from_user() fetches '1' → gpiod_set_value_cansleep(led, 1)
        ↓
GPIO pin goes high → LED on
        ↓
myled_write() returns count (1) → kernel returns it to the app
```

If the driver returns a negative code (e.g. `-EINVAL`), the app gets `-1` and the positive error code is stored in `errno`.

Program structure

Split the program into small functions, one job each:

Function	Job
`raw_open()`	Wraps `syscall(SYS_openat, ...)` to open `/dev/myled`
`led_cmd(cmd)`	Opens the file, writes one command character, closes it; returns `0` on success or `-errno` on error
`led_get()`	Opens the file, reads the state, closes it; returns `0` or `1`
`main()`	Runs the test scenario: read initial state, turn on, read, turn off, read, then send an invalid command to observe the error

6.2 User-space app that controls the myled module through ioctl

Goal

Write a user-space program that sends control commands to the driver in kernel space using **ioctl**.

In 6.1 the app controlled the LED by writing the characters `'0'`, `'1'`, `'2'` with `write()`. That treats everything as a byte stream, and the meaning of each character is an implicit agreement between the two sides. ioctl replaces this with explicit commands: each one has its own request code, a defined argument type, and a declared data direction.

Components

This lab adds one component compared to 6.1:

- **Shared header `myled_ioctl.h`.** Defines the request codes. Both the driver and the app `#include` it so they agree on the codes. If each side defined its own, the codes could drift apart and commands would be misinterpreted.
- **Driver side (to be modified).** Add a `myled_ioctl()` handler and register it in the `.unlocked_ioctl` field of `myled_fops`. The driver currently has no ioctl callback, so without this change the kernel returns `ENOTTY`.
- **App side (to be written).** Open `/dev/myled` and call `ioctl()` with the codes from the shared header.

The bridge between app and driver is still the VFS.

ioctl request code layout

A request code is a 32-bit number made of four fields:

Field	Meaning	In this lab
type (magic)	Driver identifier, so codes do not collide with other drivers	`0xE7`
nr	Command number within the driver	`1`, `2`, `3`
size	Size of the argument type	4 bytes (`int`)
dir	Direction: none, user → kernel, kernel → user	Per command

Defined commands:

Command	Macro	Direction	Meaning
`MYLED_IOC_GET_STATE`	`_IOR`	kernel → user	Driver returns the LED state to the app
`MYLED_IOC_SET_STATE`	`_IOW`	user → kernel	App sends `0` or `1` to the driver
`MYLED_IOC_TOGGLE`	`_IO`	none	Inverts the LED state, no argument

Direction is named from the app's point of view, so "read" (`R`) means the app reads data from the driver.

Flow of a command

Example: the app turns the LED on with `SET_STATE`.

```
App: ioctl(fd, MYLED_IOC_SET_STATE, &on)      (on = 1)
        ↓  (user space → kernel space)
VFS: looks up the file for fd → /dev/myled
        ↓
Kernel calls myled_ioctl(filp, cmd, arg)
        (arg is the user-space address of `on`)
        ↓
Driver: switch (cmd) → SET_STATE case
        get_user() copies the value at arg into val
        checks val is 0 or 1 → gpiod_set_value_cansleep()
        ↓
```

```
GPIO pin goes high → LED on
        ↓
myled_ioctl() returns 0 → ioctl() in the app returns 0
```

For `GET_STATE` the data flows the other way: the driver uses `put_user()` to write the value to address `arg` in the app's memory.

If the driver returns a negative code (e.g. `-EINVAL`, `-ENOTTY`), the app gets `-1` and the positive error code is stored in `errno`.

Why get_user / put_user

`arg` is a user-space address. The kernel must not dereference it directly: the address may be invalid or malicious and could crash the kernel. These helpers safely copy a small value and validate the address. If the address is invalid they return non-zero, and the driver returns `-EFAULT`.

6.3 User-space apps that work with /proc

Goal

Write two user-space programs that work with `/proc`:

- **Program 1** only reads existing kernel entries (`/proc/uptime`, `/proc/meminfo`), to get familiar with how `/proc` works.
- **Program 2** writes to `/proc/myled`, an entry created by your own driver, to control the LED and read its state back.

In 6.1 and 6.2 the app talked to the driver through `/dev/myled` (with `write`/`read` and `ioctl`). This lab adds a third channel: `/proc`.

/proc vs /dev

`/proc` is a virtual (pseudo) file system called **procfs**. Its files are not stored on disk. When the app reads a file, the kernel runs a function that generates the content at that moment, so the content is always up to date.

	`/dev/myled`	`/proc/myled`
Main purpose	Control the device	Inspect status, tune simple parameters
Content	Byte stream or ioctl commands	Human-readable text

	`/dev/myled`	`/proc/myled`
Callback table	`file_operations`	`proc_ops`
Created with	`misc_register()`	`proc_create_data()`

The advantage of `/proc` is that it works directly from the shell (`cat`, `echo`) without writing a program, which makes quick checks and debugging easy.

Components

- **Kernel side (provided).** Existing entries such as `/proc/uptime` and `/proc/meminfo`. Nothing to change, read only.
- **Driver side (to be modified).** Add the `/proc/myled` entry:

Function	Job
`myled_proc_show`	Prints the content (`state: 0` or `state: 1`) with `seq_printf()`
`myled_proc_open`	Runs when the file is opened; attaches the show function and the private data pointer to a `seq_file` (a kernel helper for printing /proc content that handles buffering and read position)
`myled_proc_write`	Receives the string `"0"` or `"1"` from the app and drives the LED
`myled_proc_ops`	The callback table that groups the functions above

- **App side (to be written).** Two programs: one uses `fopen`/`fscanf` to read, the other uses `open`/`write`/`read` to write.

The bridge is still the VFS. The difference is that the VFS routes calls to the callbacks in `proc_ops` instead of `file_operations`.

Flow of program 1 (reading an existing entry)

```
App: fopen("/proc/uptime") → fscanf(...)
        ↓  (libc calls openat, then read)
VFS: recognizes the file belongs to procfs
        ↓
Kernel runs the uptime handler → builds the string "12345.67 9876.54"
        ↓
The string is copied into the app's buffer
        ↓
App: fscanf parses two floating-point numbers → prints them
```

This program does not need `sudo` because these files are world-readable.

Flow of program 2 (writing to /proc/myled)

Example: the app turns the LED on.

```
App: write(fd, "1", 1)
        ↓  (user space → kernel space)
VFS: looks up the file for fd → /proc/myled
        ↓
Kernel calls myled_proc_write()
        ↓
Driver: kstrtoint_from_user() converts "1" to the integer 1
        checks it is 0 or 1 → gpiod_set_value_cansleep()
        ↓
GPIO pin goes high → LED on
        ↓
myled_proc_write() returns count → app gets the number of bytes written
```

To read the state back, the app opens the file and calls `read`. The kernel runs `myled_proc_show`, which prints `state: 1` into the buffer.

If the driver returns a negative code (e.g. `-EINVAL`), the app gets `-1` and the positive error code is stored in `errno`, just as in 6.1 and 6.2.

Why kstrtoint_from_user

Revision #3
Created 2026-10-19 08:16:45 UTC by trungnguyen
Updated 2026-10-19 11:07:38 UTC by trungnguyen